# HLD Level 2.4

**Design a Telemetry Ingestion System — Complete Interview Question & Answer**

**Interview question: Design a telemetry platform that collects metrics from 500,000 devices, stores historical telemetry, exposes the latest device state, supports dashboards and queries, and remains reliable during traffic spikes, downstream failures and mass reconnects.**

## 1. Requirements

**Functional:** receive telemetry, store historical measurements, expose current/latest state, query by device and time range, detect abnormal/offline devices, and support dashboards/aggregations.

**Non-functional:** high write throughput, horizontal scalability, high availability, durability, near-real-time visibility, bounded resource usage, burst tolerance and failure isolation. Strong global consistency is normally unnecessary for routine telemetry.

## 2. Capacity Estimation

```
500,000 devices
1 event / 10 sec/device
1 KB/event

Events/sec = 500,000 / 10 = 50,000
Bandwidth ~= 50 MB/sec
Events/day = 50,000 x 86,400 = 4.32 billion
Raw storage ~= 4.32 TB/day
30 days ~= 129.6 TB raw
Replication x3 ~= 389 TB before compression/index overhead
```

**This scale immediately rules out a single gateway and a single traditional database instance.**

## 3. High-Level Architecture

```
DEVICES
|
v
Load Balancer
|
v
Device Gateways
|
v
Message Broker
|
+-------+-------+
v v v
Worker Worker Worker
| | |
+-------+-------+
|
+----------+----------+
v v
Historical Store Latest-State Store
| |
+----------+----------+
v
Query API
|
v
Dashboard
```

## 4. Why a Gateway Layer?

Devices should not connect directly to storage. Gateways handle authentication, protocol termination, persistent connection management, validation, rate limiting, message-size limits and basic buffering/batching.

## 5. Device Communication

Depending on the product, devices may use HTTP, MQTT, TCP or gRPC. Persistent connections reduce repeated setup overhead but require connection state, heartbeats, timeouts, reconnection handling, file descriptors and buffers.

## 6. C++ Gateway Design

```
Many device sockets
|
v
epoll/event loop
|
v
message parsing
|
v
bounded queue
|
v
worker pool
|
v
broker producer
```

For tens of thousands of connections per gateway, event-driven I/O can avoid dedicating one blocked OS thread to every connection.

## 7. Why a Message Broker?

Writing synchronously from gateway to database couples connection handling to storage latency. A durable broker decouples ingestion from storage, absorbs temporary bursts, supports partitioning and allows consumer replay.

```
Bad: Device -> Gateway -> DB

Better: Device -> Gateway -> Broker -> Workers -> Storage
```

## 8. Burst Handling

```
Normal = 50k events/sec
Reconnect/backfill burst = 200k/sec
Consumers = 80k/sec

Backlog growth = 120k events/sec
```

The broker buys recovery time but does not solve permanent overload. Consumer capacity, retention and producer controls still matter.

## 9. Partitioning

A natural broker partition key is device_id.

```
partition = hash(device_id) % N
```

This distributes devices across partitions and can preserve ordering for a particular device without requiring expensive global ordering across the entire fleet.

## 10. Hot Devices / Noisy Neighbors

A malfunctioning device may send far above its expected rate and create a hot partition or gateway overload. Enforce authentication, per-device rate limits, quotas and maximum message sizes.

## 11. Processing Workers

Consumer workers validate, normalize, enrich, aggregate and batch telemetry before writing storage. Consumer groups allow horizontal processing scale.

## 12. Batch Writes

At 50,000 events/sec, one database operation per event is inefficient. Workers can batch hundreds of records per write to reduce network/transaction overhead, accepting a small latency trade-off.

## 13. Two Different Storage Requirements

**Historical telemetry:** queries such as CPU usage for device 123 over 24 hours. Natural key is device_id + timestamp and a time-series-oriented storage model.

**Latest state:** queries such as what is device 123 doing now? Maintain device_id -> latest state in a fast key-value/current-state store.

```
Historical:
device123 10:00 CPU=60
device123 10:01 CPU=65
device123 10:02 CPU=80

Latest:
device123 -> CPU=80
```

**History and current state have different access patterns, so separating them prevents dashboards from scanning historical telemetry to answer a current-state query.**

## 14. Historical Partitioning

Historical telemetry can be organized by device plus time bucket. Queries for a device over a bounded time range then touch only relevant partitions/ranges.

```
device123 / 2026-10-17
device123 / 2026-10-18
```

## 15. Retention and Downsampling

```
0-7 days -> raw telemetry
7-30 days -> minute aggregates
30-365 days -> hourly aggregates
```

Long-term analytics often does not require every raw sample. Downsampling can retain average/max/min/count while greatly reducing storage.

## 16. Hot and Cold Storage

Recent data can remain in a fast time-series store while older raw/aggregated telemetry moves to cheaper object storage. Retention tiering is an economic as well as technical design decision.

## 17. Latest-State Record

```
device123 -> { CPU=70%, Memory=55%, Temp=65C, Version=2.3.1, LastSeen=10:01:20 }
```

Dashboards and monitoring services can retrieve current state directly rather than scanning history.

## 18. Out-of-Order Events

```
B: timestamp 10:01:20 arrives first
A: timestamp 10:01:10 arrives later
```

Blindly applying A after B would move latest state backward. Update current state only when the incoming event has a newer timestamp/version/sequence according to the chosen ordering semantics.

```
if incoming.timestamp > stored.timestamp -> update latest state
```

## 19. Duplicate Events

At-least-once delivery can produce duplicates when an ACK is lost and a device/gateway retries. Use event_id or device_id + sequence_number when deduplication is required. Idempotent processing is often more realistic than claiming end-to-end exactly-once.

## 20. Offline Detection

Maintain device_id -> last_seen. If no valid telemetry/heartbeat arrives within an appropriate threshold, mark the device offline.

```
expected interval = 10 sec
example offline threshold = 30 sec
```

The threshold must account for jitter, packet loss, sleep behavior and processing delay rather than being chosen arbitrarily.

## 21. Offline Detection at Scale

Avoid repeatedly scanning the entire fleet when possible. Expiration buckets, timing wheels, priority queues or scheduled partitions can organize devices by their next expected deadline.

```
10:00:30 bucket -> device A, device B
10:00:31 bucket -> device C
```

## 22. Reconnect Storm

After a network outage, hundreds of thousands of devices may reconnect simultaneously. Clients should use exponential backoff plus jitter so connection establishment is distributed over time.

```
disconnect -> retry delay -> exponential backoff + random jitter -> reconnect
```

## 23. Backpressure

If gateway input exceeds broker publication capacity, bounded queues prevent memory exhaustion. Apply flow control, rate limits, bounded waiting/rejection or priority-aware sampling according to telemetry importance.

**Never allow an unbounded in-memory queue to hide sustained downstream overload.**

## 24. Device-Side Buffering

A device can keep a bounded local telemetry buffer during connectivity loss and upload it after reconnect. The buffer needs a limit and retention policy; older samples can be discarded or aggregated if necessary.

## 25. Priority and Graceful Degradation

```
Critical hardware error > temperature warning > normal CPU sample
```

During overload, preserve high-value events while sampling or dropping routine metrics. This keeps the system useful under stress.

## 26. Query Paths

```
Historical query:
Dashboard -> Query API -> Time-Series Store

Current state:
Dashboard -> Query API -> Latest-State Store
```

## 27. Fleet Aggregations

Queries such as average CPU by region should not repeatedly scan billions of raw events. Stream processors can maintain precomputed minute/hour aggregates.

```
Raw events -> Stream Processor -> avg CPU/region/minute
-> error count/model/hour
-> offline count/region
```

## 28. Reliability and Idempotency

If a worker writes storage and crashes before acknowledging the broker message, the event may be redelivered. Storage updates should therefore be idempotent where practical, using event IDs, sequence numbers or conditional updates.

## 29. Gateway Failure

Load balancers remove unhealthy gateways. Devices reconnect to healthy instances, preferably with jitter/backoff to prevent synchronized reconnect pressure.

## 30. Storage Failure

```
Storage slows -> consumers slow -> broker lag rises
```

Ingestion can continue while broker retention has capacity. Consumer lag is therefore a critical health signal and should have alerts before retention limits are reached.

## 31. Observability

Track connected devices, events/sec, bytes/sec, active sockets, gateway CPU/memory, connection failures, reconnect rate, broker lag, consumer throughput, storage latency, invalid/duplicate events, latest-state update latency, offline device count and end-to-end telemetry delay.

```
generated 10:00:00
available/queryable 10:00:04
end-to-end delay = 4 sec
```

## 32. Gateway Memory Budget

```
100,000 queued messages x 1 KB ~= 100 MB payload
```

Real usage also includes queue/container metadata, socket buffers, parser objects, thread stacks and broker-client buffers. Queue capacity must therefore be explicitly budgeted.

## 33. C++ Ownership and Copies

```
Bad: Socket -> COPY -> Parser -> COPY -> Queue -> COPY -> Worker

Better: Socket -> owned buffer -> move -> queue -> move -> worker
```

Use clear ownership and move semantics where profiling shows copying is significant. Do not add complexity without measuring.

## 34. Complete Architecture

```
DEVICES
|
v
Load Balancer
|
+-------------+-------------+
v v v
Gateway 1 Gateway 2 Gateway N
| | |
+-------------+-------------+
v
MESSAGE BROKER
|
+-------------+-------------+
v v v
P0 P1 PN
| | |
+-------------+-------------+
v
Consumer Group
|
+-------------+-------------+
v v v
Worker 1 Worker 2 Worker N
| | |
+-------------+-------------+
|
+--------------------+--------------------+
v v v
Historical Store Latest-State Store Aggregates
| | |
+--------------------+--------------------+
v
Query API
|
v
Dashboard
```

## 35. Common Interview Follow-Ups

**Why a broker?** Decouple gateways/storage, absorb temporary bursts, replay and scale consumers.

**Why partition by device?** Parallelism plus per-device local ordering.

**What about a hot device?** Per-device rate limit/quota and validation.

**Why separate latest state?** Current-state queries should not scan history.

**How handle out-of-order telemetry?** Conditional update using timestamp/version/sequence.

**How handle duplicates?** Event IDs/sequence numbers and idempotent processing where required.

**How handle mass reconnect?** Exponential backoff + jitter.

**What if consumers fall behind?** Monitor lag, scale/fix bottleneck, apply backpressure/degradation before retention is exhausted.

## 36. Interview-Ready Spoken Answer

"I'd start with capacity. Five hundred thousand devices sending one kilobyte every ten seconds gives roughly fifty thousand events per second, fifty megabytes per second and about 4.32 terabytes of raw telemetry per day. Devices connect through horizontally scalable gateways responsible for authentication, connection management, validation and rate limiting. Gateways publish to a partitioned durable broker rather than synchronously writing storage, which

decouples ingestion from downstream processing and absorbs temporary bursts. I'd partition primarily by device ID so processing scales while events for a device retain local ordering. Consumer workers validate and batch writes. Historical telemetry goes to time-series-oriented storage, while the latest state is stored separately as device ID to current state. I'd use retention/downsampling for older data. The design also handles duplicates, out-of-order events, hot devices, reconnect storms and backpressure using IDs/sequence numbers, conditional latest-state updates, per-device quotas, reconnect jitter and bounded queues. For a C++ gateway handling many persistent connections, I'd consider epoll-based I/O feeding bounded worker queues rather than one thread per connection."

## 37. Five Decisions to Remember

1. **Many persistent devices -> GATEWAY LAYER.**
2. **High write throughput / downstream decoupling -> MESSAGE BROKER.**
3. **Parallelism + local order -> PARTITION BY DEVICE ID.**
4. **Historical queries differ from current-state queries -> TIME SERIES + LATEST-STATE STORE.**
5. **Slow downstream or reconnect bursts -> BOUNDED QUEUES + BACKPRESSURE + JITTER.**

## 38. Final Revision Flow

```
Requirements
-> capacity
-> device protocol/gateways
-> broker
-> partition by device
-> consumers/batching
-> history + latest state
-> retention/downsampling
-> duplicates/out-of-order
-> offline detection
-> reconnect storms
-> backpressure
-> reliability
-> observability
-> C++ epoll/worker/ownership deep dive
```